

LibSeeker: Compilation Unit–Aware Library Identification for Statically Linked Binaries

Abstract—Identifying third-party libraries in statically compiled binaries is essential for reverse engineering, vulnerability detection, and software supply chain security. However, existing approaches often operate at the function or basic block level, making them highly sensitive to compiler optimizations and code transformations, which degrades accuracy. We present LibSeeker, a library identification system that operates at the granularity of Compilation Units (CUs), mirroring how compilers and linkers integrate object files from static archives and thereby overcoming the limitations of function- or basic-block-level analysis. LibSeeker captures both the semantic and syntactic characteristics of binary code through a multi-stage pipeline. It first derives instruction-level embeddings from raw assembly code, which are integrated into a graph matching network specifically designed for comparing function-level Control Flow Graphs (CFGs). The network generates function embeddings that are compared using cosine similarity. Finally, a novel CU-level matching algorithm determines whether a compilation unit and, thus, its corresponding library are present. We conduct an extensive evaluation on a dataset of binaries compiled with multiple compilers and optimization levels, covering 75 distinct libraries. LibSeeker accurately detects library inclusion, achieving an F1 score of 0.79 and outperforming existing baselines. We also evaluate its performance on the more challenging task of library version identification and discuss its current limitations.

Index Terms—Library Identification, Binary Analysis, Reverse Engineering, Vulnerability Detection

1. Introduction

The problem of binary diffing has been faced over the past 20 years without reaching a conclusive solution. The core of this problem is determining whether two compiled pieces of code are logically identical, despite any differences that may be found. More properly, we want to establish if they come from the same origin, i.e., the same source code, independently of how it has been compiled in the latter stages, which, theoretically, is undecidable as it can be reduced to the halting problem [1]. However, two main approaches have been developed: one involves binary diffing, namely attempting to statically identify if two executable files originate from the same source, while the other involves identifying third-party code reuse in already compiled software. In this work, we focus on this second aspect, attempting to identify library functions imported into statically linked binaries.

Several approaches and solutions have been proposed over the years. Initially, several studies relied on the

manual extraction of useful features for each function, allowing for fair comparisons to be made. Follow-up approaches migrated toward graph isomorphism methodologies, exploiting the complexity of graphs to facilitate more comprehensive information analysis. Graphs are capable of carrying both semantic and syntactic information for each function; therefore, more precise identification can be enforced. With the introduction of machine learning and neural network approaches, a definitive change has been introduced: effective, fully automated embedding generation models have spread thanks to their capability of encoding information into meaningful, high-dimensional vector representations. Examples such as Gemini [2], Genius [3], and Graph Matching Network [4] are the first approaches that utilize a graph neural network to generate an embedding for functions, using as input an attributed control flow graph where each node has attributes based on manually selected features of functions. Approaches such as PalmTree [5] or Asm2Vec [6] improve the state of the art by substituting manual selection with the automated extraction of information from raw assembly, using a BERT-based model [7] fine-tuned to process assembly instructions rather than natural language. Li et al. [5] prove that it is possible to reach better results thanks to a more efficient, condensed representation of the semantics defined by instructions. Despite promising results, all these prior works have focused on a function-level comparison, seeking correspondences among functions to identify them as imported from a target library. This level of granularity is too fragile, considering the scope of comparison is too restricted; moreover, it generates a high number of false positives, as the semantics and syntax of small functions are dominated by their epilogue and prologue, which are generated by compilers, resulting in unreliable features to count on.

This paper presents LibSeeker, a novel approach rooted around **Compilation Units (CUs)** that overcomes the shortcomings of existing library identification methodologies. Our goal is to mimic how static compilation works in terms of compilation units import, while leveraging the benefits of embedding generation models. Our pipeline of analysis constructs functions' embeddings directly from raw assembly, combining Control Flow Graphs (CFGs) structural information with the semantics carried by instructions. These function embeddings are then used as a mathematical representation to which the cosine distance can be applied. Specifically, we design an ad-hoc searching algorithm to check whether a compilation unit has been imported, thereby identifying the corresponding imported library.

To validate LibSeeker, we directly address the limitations of existing works: the lack of publicly or well-

documented benchmarking datasets for fair comparison, and evaluations restricted to a small set of popular libraries. We therefore build a comprehensive benchmarking dataset and design an exhaustive experimental campaign that (i) demonstrates the validity of the compilation unit as a reference for library identification, (ii) evaluates its performance in the library identification task, and (iii) assesses its robustness across different library versions and compilers. LibSeeker achieves an F1 Score of 0.79 on library identification aligned with state-of-the-art performance, indicating that Compilation Unit granularity enables precise matching of imported libraries. We further evaluate version identification—a substantially more complex problem—where the expected drop in performance still confirms the promise of LibSeeker under diverse compilation settings.

In summary, the contributions are the following:

- We propose LibSeeker, a new approach for library identification in statically compiled binaries, based on compilation units.
- We conduct extensive empirical evaluations and demonstrate that LibSeeker yields valid and statistically significant results, proving its capabilities in the library function recognition task when considering statically compiled binaries.
- We release a publicly available dataset constructed from well-known open-source binaries, covering a diverse range of binary types, functionalities, and library dependencies. We further support reproducibility and follow-up research by open-sourcing LibSeeker together with the accompanying datasets and trained models used in our experiments¹.

2. Background

In this section, we provide the background needed to understand our approach to library identification in statically linked binaries. We begin by clarifying the objective and inherent challenges of the library identification problem, with a focus on the differences between dynamic and static linking, and then summarize the compilation and machine-learning concepts that underpin our method. Specifically, to address the *library identification task* carried out at CU granularity in *statically* linked software, we adapt *word embeddings* [8] and *Graph Neural Network (GNN)* [9].

2.1. The Library Identification Task

Library identification is the process by which, given a binary, we would like to identify the libraries it depends on, whether static or dynamic. As for dynamic dependencies are concerned, they can easily be identified since all the information can be statically retrieved from the binary’s header. These pieces of information must be available for the loader to correctly load everything into the virtual process memory when the executable is launched; therefore, a simple static analysis would be sufficient.

On the other hand, when libraries are statically linked within the final executable, the identification process deals

with more complex challenges, especially when symbols are missing (i.e., stripped). In this case, the compilation process often results in very large files, where functions have undistinguishable, but different origins: either the source code of the project or the libraries on which the project depends (eventually in a recursive manner, since libraries also have some other dependencies on their own).

Moreover, library functions are not imported as standalone; they come together with all the functions of the same compilation unit, thereby enlarging the code base that a reverse engineer should analyze even further. In other words, library identification in statically compiled binaries means to correctly identify the origin of each function: either the source code itself or the correct library to which such a function belongs.

Many previous works have attempted to provide useful support; however, many of them suffered from significant limitations due to an approach based on function-to-function pattern matching, which reduces flexibility (pattern matching is not fault-tolerant). At the same time, this technique increases the search space for matches due to the high number of comparisons required.

2.2. Dynamic and Static Compilation

Compilation is the process through which source code files are translated into an executable binary, carried out through a sequence of transformations performed by the compiler and linker. During this process, various flags can be specified to set optimization levels and instruct the linker on how to handle external dependencies such as libraries. Software code bases are typically organized into separate source files to improve maintainability and streamline development, allowing for a clear and logical division of functionality. All these source files are ultimately merged into a single executable during the compilation process, which, as a result, inherits all the external dependencies. When dealing with libraries, these source files are referred to as **compilation units**.

Let us suppose the `dynamic` compilation flag is set. In that case, external dependencies (i.e., dynamically imported libraries) are linked in such a way that when the program is launched, the loader checks the headers of the binary file, so as to load all the external libraries into the process’s virtual memory, but into different pages of memory from those of the binary. For this case, the linker will have references to dynamic libraries, specifically to `.so` files (i.e., shared object files), whose specific structure allows dynamic symbol resolution at runtime. Despite the subset of actually referenced functions and symbols, all those defined in the various compilation units are loaded into memory since they are contained in the shared object file. In terms of size, the result of this process consists of small compiled binaries, as they contain only the functions derived from the source code, along with the necessary overhead to dynamically resolve external symbols at runtime.

Instead, let us suppose the `static` compilation flag is set. In that case, the way of proceeding is a little different: at compilation time, per se, there are no changes since all the source files will be compiled; what changes is the mechanism by which symbols are resolved, because it must happen at compilation time rather than at runtime,

1. <https://anonymous.4open.science/r/LibSeeker-F35D>

thus it happens statically. The linker is provided with all the needed libraries (i.e., all the external dependencies) that solve at least one of the unresolved symbols after the compilation phase. For this purpose, static libraries are defined as `.a` files (i.e., archive files), which are archives of object files. In fact, compilation units are not fused in a single file, as for dynamic libraries; instead, they are collected in such an archive, so that the linker can explore it, choosing all and only the compilation units to import, depending on the symbols each CU resolves. With such a process, in the end, there will be only one large binary file, self-contained in terms of undefined symbols, for both variables and functions. At runtime, when the executable file is launched, the loader loads only this single file into virtual memory, without loading any additional libraries, since everything has already been resolved statically during the linking stage. The produced binary file is much larger, since, alongside the code from the actual source files, it also contains all the code coming from the imported compilation units (thus, indirectly from external libraries): in such a case, static analysis is much more complex, given the much larger code base to be analyzed. We can conclude that, given how compilers and linkers work, the smallest, indivisible unit for imported static libraries is the **compilation unit**. During the compilation stage, it either resolves at least one symbol, forcing it to be included as a whole in the final executable, or it is ignored, since it is not relevant to the current executable.

A final consideration should be made regarding Link Time Optimization (LTO), a highly aggressive but effective optimization technique that affects how binaries are compiled. With the proper compilation flag set, source files are translated into a bitcode object file, which is then passed to the linker along with other object files. This intermediate representation of object files enables compilers to combine them into a highly optimized binary file, ready for execution. The optimization introduced here has a significant impact on the structure of the final executable, posing additional challenges for reverse engineering tasks. In the scope of this work, LTO is not considered, though.

2.3. Machine Learning for Binary Analysis

With the rise of machine learning models, binary analysis has been pushed to a new frontier, which is no longer limited to manual static (or dynamic) analysis. More scalable techniques have been developed and adapted to specific binary-only analysis to capture as much information as possible, while condensing it into efficient and compact notation, enabling automated comparison and classification. While the underlying theory can be found in Appendix B, this section presents how such models have been adapted for binary analysis tasks.

Word Embeddings. Assembly language is a low-level programming language that serves as a symbolic representation of machine code. Since it is more human-friendly, it is easier for reverse engineering to understand and analyze when the source code is not available. With a similar approach to natural language [8], it might be helpful to capture semantic and syntactic relationships among assembly instructions (analogously to words and sentences for natural language). It is important, though, to

keep in mind that each assembly instruction is going to have one or more operands, and the relationships among instructions and operands are both syntactic, in terms of the sequence of instructions, and semantic, in terms of how each instruction affects the execution flows. For example, two instructions might be far apart in terms of residency location, but they may be executed one after the other due to a jump within the CFG. With respect to natural language processing, other than the semantic meaning of instruction, it is important to consider the syntactic relationships among *words*—i.e., instructions—as well. With proper adaptations of BERT [7] or Word2Vec [10] and specific training and fine-tuning, it is possible to adapt neural network models to cope with assembly instructions and their structure, thereby generating meaningful embeddings. The extent to which such embeddings refer can vary, depending on the model and the final scope: embeddings can refer to single instructions, to basic blocks, or even to entire functions. Asm2Vec [6] (Word2Vec-based model) and PalmTree [5] (BERT-based model) represent the current state of the art, and they are largely used to build useful compact representations of assembly instructions, used for tasks such as binary diffing or code similarity.

Graph Neural Network. Given a function, the syntactic meaning of assembly instructions can be represented by means of a Control Flow Graph, which compactly embeds all the possible execution traces. Yet, even assuming that reconstructing the CFG can be done easily and perfectly, comparing two different graphs to assess whether two functions are the same remains a very complex task. The graph isomorphism technique has been applied since its introduction in 2006 with Bindiff [11], but it is not capable of fully embracing all the possibilities that can be generated by compilers: different tools and compilation flags can lead to deeply different structures despite the same source code. Recent works have shifted the focus to adapting machine learning techniques to work with graphs as inputs. The goal is to characterize each node with the semantic information of the assembly instructions it contains and combine this information with the syntactic information of functions, thereby resembling the structure of the reconstructed CFG. Considering the current literature, several studies were developed: CFG2Vec [12], which has been inspired by the methodology of Word2Vec [13]; Genius [3], which is a bug search engine that leverages converting CFG to high-level numeric feature vectors. Other works, such as Gemini [2] and Graph Matching Network [4], proposed a Graph Neural Network (GNN) [9] structured as a Siamese Architecture [14]. This particular network design intrinsically requires pairs as input: each element is processed through the same, identical neural network, and then both outputs are evaluated on their distance (or equivalently, their similarity); thus, these works can easily be adapted for code similarity analysis. Summarizing, the scope of such tools is to generate compact and efficient representations for graphs, combining both their semantics and structure.

3. Problem Statement and Motivation

Binary-only code analysis is a crucial activity for many software security tasks, including reverse engineer-

ing [3], [15]–[18], vulnerability discovery [19]–[24], and malware analysis [11], [25]–[28]. Despite decades of research, it remains an open problem: automated tools and machine learning approaches achieve notable results [5], [6], [29], but their performance degrades significantly under aggressive compiler optimizations and function inlining [30], [31]. The recent adoption of large language models (LLMs) for binary analysis tasks, such as code similarity, summarization, and function name or type recovery [32]–[34], has shown promising results, but empirical studies consistently report substantial limitations on stripped or highly optimized binaries, indicating that their effectiveness for reliable function identification is still far from satisfactory.

Within this broader context, library identification provides valuable support to downstream analysis tasks by recovering the external dependencies of an executable and determining which portions of a statically linked binary originate from third-party libraries. This information benefits several activities. For example, it improves reverse engineering tasks by enabling analysts to concentrate their attention on application-specific code rather than repeatedly examining common library implementations. It also strengthens software security assessments by enabling analysts to detect vulnerable components when they recover a specific library version with known security issues. At the same time, it delimits the non-vulnerable library code, allowing further vulnerability analysis to focus more effectively on the application-specific regions of the binary. In fact, if we focus on the structure of a statically linked binary, we have a more formal mathematical treatment of the library identification problem. At its core, this task aims to detect small, coherent communities within a larger collection of objects. Given the set of binary functions (the universe of our parallelism), we want to detect the subsets of functions belonging to the same compilation unit (the communities to be detected). Once all communities are detected, they can be easily mapped to the respective libraries, thereby defining which libraries should be marked as imported. Solving this problem in the context of binary-only analysis is very complex, since function origins are indistinguishable once the binary is compiled.

Prior works have combined static analysis approaches with machine learning techniques, aiming to extrapolate and combine as much information as possible. However, these methods are all constrained by their reliance on function-to-function matching. This perspective fails to capture how functions are actually brought into a binary during static linking: they are not imported as standalone entities, but rather together with the compilation unit to which they belong. The strength of our work stems from mimicking how compilers operate: comparisons are performed considering compilation units as atomic entities (once again, as a small, atomic community), either imported or excluded, allowing us to enforce and exploit all available information, resulting in highly accurate results.

The state of the art in this area primarily focuses on the Android world, where third-party code reuse is a major concern, particularly from a security perspective. Yet, the very same problems can be extended to software development for other operating systems (e.g., Linux, Windows, macOS), where significantly fewer studies are available.

Research Goal. The goal of this paper is to enable reliable identification of static libraries in statically linked binaries—ideally down to the exact version—while remaining robust to differences introduced by compilers and optimization settings. To this end, we propose, to the best of our knowledge, the first approach that models and searches for imported **Compilation Units** within statically linked binaries, mirroring how compilers and linkers operate under the `-static` flag. Concretely, we aim to (i) map functions into a high-dimensional embedding space using graph neural networks over their control flow graphs so that imported compilation units can be retrieved via embedding similarity, and (ii) mark the corresponding libraries as dependencies of the analyzed binary when such matches are found. To support this goal, we also construct and release a comprehensive, representative dataset of well-known open-source binaries, providing a common benchmark for future evaluation and comparison.

4. LibSeeker

The key novelty of LibSeeker lies in performing library identification at the Compilation Unit (CU) level, rather than at the function or basic block level as in prior works. This design decision is motivated by how static libraries are actually compiled and linked: as explained in Section 2, static libraries are archive files made with compilation units, which, at compile time, the linker imports as atomic entities, never as isolated functions. Consequently, approaches that operate only at the function level (or even at the basic block level) are inherently fragile, since a few misclassified functions can either lead to spurious library matches or miss entire dependencies. LibSeeker proceeds in three main stages, as summarized in Fig. 1:

- *Preprocessing.* We disassemble both the target binary and candidate libraries, extracting functions and reconstructing their Control Flow Graphs (CFGs). Static libraries are decomposed into the compilation unit they are made with, which serve as the units of comparison.
- *Embedding Generation.* Each function is encoded into a high-dimensional vector representation. To capture both instruction semantics and control-flow structure, we combine instruction-level embeddings from PalmTree [5] with a graph neural network (Gemini-like [2]) applied over CFGs. The resulting embeddings are capable of providing accurate similarity comparisons between functions even across compilers and optimization levels.
- *CU Matching.* For each candidate library, we compare its compilation units against the binary. We develop a novel CU-matching algorithm that combines cosine similarity with the Hungarian algorithm [35] to align functions between a CU and a candidate *window* of functions in the binary. A compilation unit is marked as matched if its overall similarity exceeds a given threshold; finally, a library is marked as present if a sufficient number of its CUs are matched.

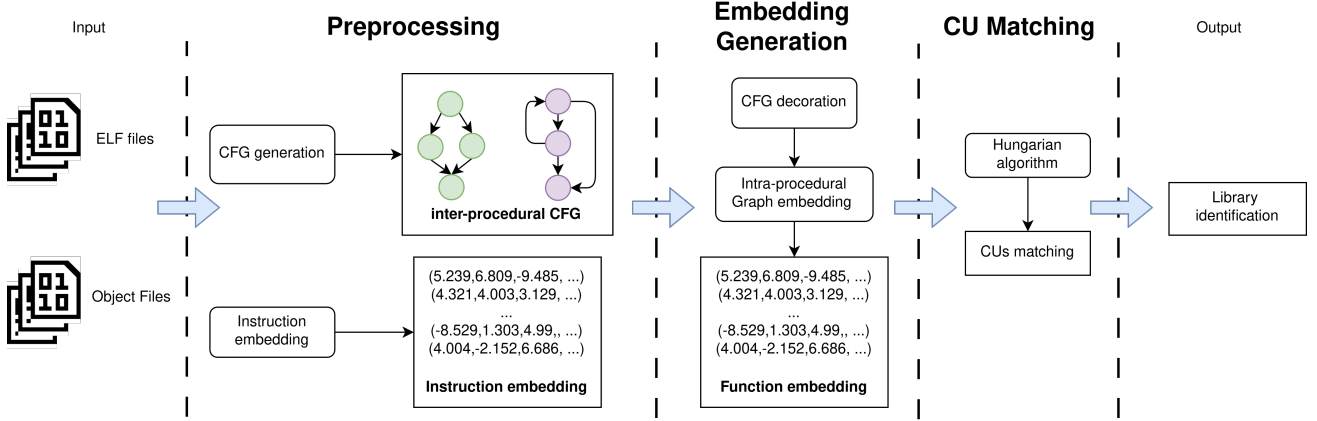


Figure 1. Architectural overview of the approach we developed. Each binary and each library is analyzed following this flow, from left to right. Ultimately, we will have a binary result: either the library is matched or not.

This CU-level strategy enforces compiler-consistent granularity, reducing the risk of false positives resulting from small or compiler-generated functions. Additionally, it provides a natural trade-off between recall (library detection) and precision (avoiding spurious matches).

4.1. Preprocessing

The first step in our pipeline is to normalize program inputs to extract function-level features, which are then fed as inputs to later stages. For target binaries, no additional preparation is required: functions can be directly identified and used as the atomic units of static analysis tasks. In contrast, libraries must undergo a decomposition step. Specifically, we extract all the compilation units from each library and treat them as self-contained components that serve as the counterparts to the target binary. This ensures that both binaries and libraries are represented uniformly. Once this normalization is complete, both binaries and library-derived units undergo the same preprocessing procedure.

The ELF and object files are properly analyzed and disassembled (in our case, we adopt Angr [18]). This first stage enables the reconstruction of function boundaries and the parsing of instructions into basic blocks. For each function, we construct its Control Flow Graph (CFG) and derive a set of structural and semantic features. In our pipeline, we consider only intra-procedural Control Flow Graphs, as outgoing edges to other functions are dropped: at this stage, our focus is on the structure and semantics of functions as standalone entities. These graphs embed the relevant structural information to generate meaningful function embeddings, which are used to measure the distance—and, by extension, the similarity—between data points in the reconstructed high-dimensional space.

Alongside the structural information given by the CFGs, the semantics of a program, or, in this case, of a function, is defined by the assembly instructions. In fact, they effectively define the sequence of operations to be executed, thus shaping the meaning of the edges in the reconstructed CFGs (Control Flow Instructions define most of the edges of a CFG). Thus, they carry a great amount of information, which reverse engineers try to extract via static and dynamic analysis techniques. Many

prior works focused especially on manually extracted features [36], but Li et al. [5] demonstrate that automated tools, together with fine-tuned natural language processing models, provide much better results. Such models can interpolate large amounts of information, providing faster and more accurate outcomes. Thus, following the methodology illustrated by Li et al., we utilized the pre-trained PalmTree model, available on Github², to generate embeddings represented as 128-dimensional vectors.

The output of this stage is a standardized collection of function-level representations, each annotated with its corresponding CFG and feature set. This provides the foundation for subsequent stages of the pipeline to generate meaningful function embeddings.

4.2. Embedding Generation

At this stage, the goal is to combine instruction-level semantics with control-flow structures: LibSeeker adopts a Graph Neural Network (GNN) capable of aggregating all the features that were previously extracted in a resulting vector representation.

Firstly, Control Flow Graphs require decoration for proper processing by our GNN, so each node needs to be characterized, while preserving locality and data dependencies across instructions. Following the approach proposed by Li et al. [5] as an aggregation function, we adopt the instruction-level average. However, this approach can lead to the loss of certain information related to instruction locality, particularly when applied to entire functions. Specifically: (i) the final embeddings tend to be very similar across different functions, resulting in a loss of data dependencies; (ii) instruction locality is ignored by the chosen aggregation function, whereas the structural information of functions' CFGs remains a crucial feature for this task.

Given these limitations, we restrict the scope to basic blocks for which these side effects are strongly mitigated and do not negatively impact the outcome. Now, each basic block—i.e., a node in the reconstructed CFG—is characterized with the average of the embeddings of its instructions. The task of generating an embedding for each function is then delegated to a GNN.

2. <https://github.com/palmtree-model/PalmTree>

We adopt Gemini [2], a graph neural network with a siamese architecture [14], that operates on Attributed CFGs, where each node has an attribute. In our design, it corresponds to the basic block characterization as just described. As demonstrated by Li et al., manual feature extraction (e.g., the number of instructions per basic block, referenced strings) is a complex and time-consuming task, where correctly tuning the weights of extracted features significantly affects the outcome, leading to a highly error-prone process. Our design, instead, relies on embeddings that are significantly more expressive, whose correct interpolation is left to our GNN. This approach provides a fully automated and scalable method for function encoding, ensuring that even large and heavily optimized binaries can be effectively analyzed.

At the end of this stage, each function is represented by a fixed-length embedding vector suitable for similarity analysis across compilers and optimization levels.

4.3. CU Matching

We leverage function embeddings to identify whether a Compilation Unit (CU) was imported into a binary. Assuming all functions and compilation units have been analyzed, we aim to identify the best pair of functions across CUs and the target binary. This pair is the starting point from which the matching algorithm proceeds. To define the *best* pair, LibSeeker leverages cosine distance [37] as a distance metric, since, as the work of Peng et al. [38] proves, high-dimensional spaces suffer from the curse of dimensionality. Cosine similarity has been demonstrated to be robust to high-dimensional embeddings, whereas other distance metrics tend to lose discriminatory power in such spaces, resulting in data points that are all equally distant from each other.

More formally, we define the cosine similarity of the pair (f_i, f_j) , according to (1):

$$\cos_sim(f_i, f_j) = 1 - \frac{\cos_dist(f_i, f_j) + 1}{2}, \quad (1)$$

where $\cos_dist(f_i, f_j) \in [-1, 1]$.

The remapping transforms the standard cosine distance into a normalized similarity score in the domain $[0, 1]$, which facilitates comparison across various function pairs.

The scope of our search is to define whether a compilation unit has been included at linking time, since, as we explained in Section 2.2, compilation unit are atomic particles of libraries which cannot be partially brought in. Consequently, if the CU has been imported, all its functions must be in the static binary under analysis.

Our search through the binary has three main steps, as highlighted by Algorithm 1. First, we compute a similarity matrix between all the functions of a CU and the target binary to obtain all the possible comparison results. Second, we generate candidate matches using a sliding window approach. Among all the generated pairs, we select the one with the highest similarity score, which we can define as an *anchor*. Then, given a compilation unit of size N , we create N consecutive windows of size N in the binary, all of which include the matched function. Such a procedure corresponds to the definition of all possible alignments of the CU functions inside the binary. For each window,

Algorithm 1 LibSeeker’s matching algorithm.

```

1: procedure MATCHCU( $B, CU, N, th, min\_size\_cu$ )
2:    $Sim\_Mat \leftarrow 0$  ▷ Stage 1
3:   for  $f_i \in Functions(B)$  do
4:     for  $f_j \in Functions(CU)$  do
5:        $Sim\_Mat[f_i][f_j] \leftarrow \cos\_sim(f_i, f_j)$ 
6:     end for
7:   end for
8:    $Best\_F \leftarrow \text{argmax}(Sim\_Mat)$ 
9:    $windows \leftarrow \{s \in slices(N, Functions(B)) \mid$ 
    $Best\_F \in s\}$ 
10:  for  $w \in windows$  do ▷ Stage 2
11:     $w\_sim \leftarrow \text{Hungarian\_Algorithm}(w)$ 
12:    if  $w\_sim > Best\_CU\_sim$  then
13:       $Best\_CU \leftarrow w$ 
14:    end if
15:  end for
16:   $avg\_sim \leftarrow \text{avg}(Best\_CU)$  ▷ Stage 3
17:  if  $avg\_sim \geq th \wedge N \geq min\_size\_cu$  then
18:     $Best\_CU$  is marked
19:  end if
20: end procedure

```

we apply the Hungarian algorithm [35]—a polynomial-time combinatorial optimization algorithm—to maximize the total average cosine similarity across the assigned function pairs. This step ensures that we select the most semantically meaningful function mapping for the CU.

Finally, to determine whether the CU has been imported, we compute the average cosine similarity of the functions in the best-matched window. If this value meets a predefined threshold, we consider the CU imported and mark its corresponding library as a dependency. Yet, we need to consider that small functions have a similarity score dominated by their prologue and epilogue, whose structure is defined by the compiler and remains unchanged across functions, except for some constants that do not affect the outcome. Thus, we introduce two hyperparameters: min_size_cu , the minimum number of functions a CU must contain to be considered reliable, and min_cu_lib , the minimum number of CUs that must be matched for a library to be considered imported. These thresholds ensure that both CU-level and library-level decisions are robust and semantically meaningful. In fact, the first threshold provides evidence that in large enough CUs, the influence of function prologues and epilogues is only minimal, enabling semantically meaningful comparisons. Whereas the second threshold allows proper tuning of the final results: increasing the number of CUs that must match before a library is marked as a dependency helps reduce false positives.

5. Function Embedding Evaluation

Before diving into the evaluation of LibSeeker as a framework in more depth, we assess the capabilities of our approach in generating embeddings correctly for each function. As described in Section 4, we reimplement the graph neural network used for function embedding, taking inspiration from Gemini [2]; thus, it is crucial to prove the correctness of our own implementation and prove that it is coherent with the findings of the original work.

5.1. Dataset Description

With respect to the approach proposed in PalmTree [5], we do not use the same dataset. The one used in the paper was not large enough to train the entire GNN, given the very high number of internal parameters, nor varied enough to capture different executable scopes and behaviors. Instead, we compile three different versions of the three well-known packages `binutils` [39], `coreutils` [40], and `findutils` [41], for a total of six projects (`binutils-2.36`, `binutils-2.40`, `binutils-2.44`, `diffutils-3.10`, `diffutils-3.11`, `diffutils-3.8`, `findutils-4.10.0`, `findutils-4.8.0`, `findutils-4.9.0`). We choose two different compilers, GCC and Clang, and use three different versions of each (GCC-12, GCC-13, and GCC-14, and Clang-17, Clang-18, and Clang-19). We compile the projects considering all possible optimization flags (from O0 to Os) in both versions, PIE and NO-PIE. The static compilation flag is not taken into consideration, as it is not relevant for training the GNN. In the end, there are 74 executables, each with 72 variants due to a unique combination of compiler version, optimization flag, and PIE flag, for a total of 5 328 unique binaries.

The goal of such a dataset is to cover as many different cases as possible in terms of CFGs and instructions, which resembles the variety of machine-code structures found in the wild. This enables our network to generalize to new, unseen data seamlessly, resulting in good performance. Yet, we cannot use all the functions from each binary, because it would result in a too large dataset, making it unfeasible to use. Therefore, to achieve a good trade-off between usability and a representative dataset, we select 50 executables at random from the 74 unique ones, for which we extract 40 different combinations of compiler version, optimization flag, and PIE/NO-PIE flag. For each of these groups of binaries, we extract a set of functions in common across all variants, selecting them based on the number of basic blocks per function. We identify five groups, varying in the number of basic blocks: very small, with 0 to 6 blocks; small, with 7 to 24 blocks; medium, with 25 to 74 blocks; medium-large, with 75 to 149 blocks; and large, with more than 150 blocks. The stratification of functions’ sizes has the scope of avoiding overfitting against some specific structure of functions, while keeping the dataset balanced. It is worth noticing that as the number of basic blocks per function increases, such discrimination of classes becomes less relevant.

In the end, we process 1 000 000 functions, considering as a positive pair of functions a pair for which each element belongs to the same executable but a different variant: the function’s source code would be the same despite a different compilation pipeline. On the contrary, any pair of functions coming from different source code results in a negative pair. To maintain a balanced dataset between positive pairs and negative pairs, we stratify our dataset by first generating all the possible positive pairs, for a total number of 500 000 pairs, and then we generate an equivalent number of negative pairs by choosing at random two functions, ensuring that they do not belong to the same project but different variants. Considering an 80/20 split, we obtain 720 000 pairs for training and 180 000 pairs for validation. For the testset, we use the same one as Li et al. to ensure comparable results and assess

whether our re-implementation follows the original. Thus, we compile the GNU libc project [42], OpenSSH [43], and OpenSSL [44] within a Docker container using Ubuntu 18.04 and GCC-9. We compile 52 binaries, for which we generate a balanced testset, following the same methodology as described earlier (i.e., with an equal number of positive and negative pairs).

5.2. Experimental Settings

For this experiment, we test our graph neural network in the same configuration as used in the latter experiments, described in Section 6. We keep the embedding size of PalmTree [5] as 128; on top of that, there is our GNN with output dimension as 128, and an internal dimension of 512. The reason for such large dimensions lies in the complexity of the functions’ structures we want to capture: a network too small would not be able to generalize enough, whereas a network too big would affect the usability, causing the distance of any two data points not of interest anymore (making all vectors distant from each other due to a too high-dimensional space). Following the extrinsic evaluation proposed by Li et al. [5] (the most similar test to our purpose), we test our GNN with labeled pairs of functions, assessing whether the model can correctly classify functions. In particular, as labels, we assign the expected value of the cosine distance between the functions within the same pair: we assign the label ‘1’ to positive pairs (the resulting embeddings are parallel vectors), and assign the label ‘0’ to negative pairs (the resulting embeddings are orthogonal vectors). The minimum value ‘-1’ was not considered, as the resulting embeddings would be anti-parallel, which, from a geometric point of view, would result in compressing all the embeddings into a single direction. At last, for training, we select 175 epochs, with a learning rate of 0,0001.

5.3. Experimental Results

To evaluate our model against the results claimed by Li et al. [5], we used the same testset (described earlier in Section 5) to recreate the same environment, for a valid comparison. As we can see from the ROC-AUC plot, reported in Fig. 2, and from the AUC values obtained with the testset (see Table 1), the results we obtain with our model are even better than those reported in the PalmTree original paper. The underlying reason for this advantage comes from the training task of the GNN. We utilize a much wider dataset, with higher variance, that considers different compilers and compilation flags. Such a training set enables our GNN to generalize more effectively to new, unseen data. Moreover, we also verify how pairs of functions are evaluated by our model, obtaining coherent results for the geometric model we create: if the two functions are equal to each other, the resulting cosine similarity is a value above 0.9—the resulting embeddings are parallel or almost parallel. When two functions are different from each other, the resulting cosine similarity is around 0.5—the resulting embeddings are orthogonal to each other. These results are consistent with the transformation we applied, as shown in (1).

We also try to compare our results against other models, such as `Asm2Vec` [6], but unfortunately, no official

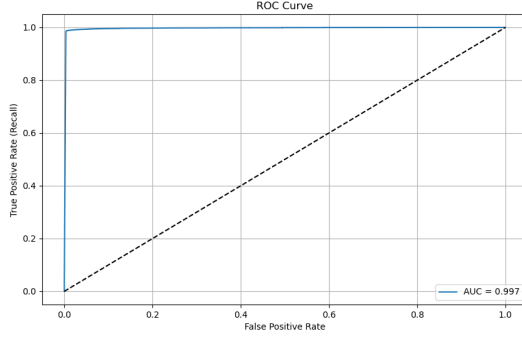


Figure 2. ROC-AUC curve.

TABLE 1. AUC VALUES REPORTED FOR COMPARISON. WE REPORT THE MOST RELEVANT RESULTS FROM THE PALMTREE PAPER [5] FOR COMPARISON WITH OUR OWN.

Model	AUC	Model	AUC
Asm2Vec	0.823	PalmTree	0.921
Gemini	0.866	LibSeeker	0.997

implementation has been released; therefore, we pick one unofficial implementation³ that claims comparable results (despite the open issues on GitHub). After training and testing on the same datasets we use for our model, we find that the results are too poor to be used for any valid comparison.

6. LibSeeker Evaluation

The goal of our experiments is to demonstrate the capabilities of our tool in various scenarios, thereby proving its potential and identifying its ability to locate a library within a statically compiled executable under different environmental conditions and settings. Specifically, we want to answer the following research questions:

- RQ 1.** *How well LibSeeker performs in identifying the library imported into a static executable?*
- RQ 2.** *Given that a library is correctly identified, can LibSeeker correctly recognize the library version adopted?*
- RQ 3.** *Given that a library is correctly identified, can LibSeeker still recognize it if compiled with a different compiler?*

The first research question examines LibSeeker’s performance in the library identification task, while the second one focuses on the precision of our tool, specifically in terms of code diffing, when considering different versions of the same library, as they may have different exported functions, APIs, or even vulnerabilities. Similarly, the last research question attempts to assess how well LibSeeker can generalize when considering different compilation scenarios, including two different versions of the same compiler. Even compiling from the same source can generate significant differences in the compilation units, given the varying optimization strategies across versions of the same compiler.

3. <https://github.com/oalieno/asm2vec-pytorch>

These experiments are conducted over a wide enough dataset that could cover various and different characteristics expected to be found in binaries (e.g., text editors, cryptographic features, sockets, etc.), as well as a wide enough set of libraries to be checked as dependencies; we also considered different compilers, GCC and Clang, as well as different versions of the same compiler (GCC-9 and GCC-13).

6.1. Dataset description

We select 20 well-known, open-source projects for which we manually check their dependency: in our experiments, we want to prove if our tool is capable of recognizing different libraries, not limiting the scope of our search just to the standard C library (and a few others), which are the most common ones, therefore always present in any binary. We create a dataset that relies on 75 open-source libraries as dependencies spread over the different projects. For what concerns the binaries, we compile all those projects statically, using both GCC-13 and Clang-18, if the project supported both compilers, enabling some of the optimization flags: `O0`, as a baseline, since no optimization is carried on; `O2`, one of the most used flags, thus considering one of the most common scenarios; `O3`, since it performs very aggressive optimization; `Os`, to check if space optimization could influence how compilation units are imported during compilation. To have full control over the dataset we are building, we also download the source projects of such libraries and compile them with GCC-13, leaving the default optimization level. At last, for the second research question, we also consider three different library versions: a default one to have a baseline as a reference; one very close to the baseline (e.g., a previous, minor release version); and a much older one (e.g., a previous, major release version).

6.2. Experimental Metrics

We consider three performance metrics: precision (P), recall (R), and F1 score (the mathematical formulas are provided in Appendix C). These are the ones that can effectively capture how well our tool can identify when a library is imported. Except for the hyperparameter selection, we cannot consider accuracy (A) as well due to the high number of true negative samples: on average, binaries tend to have few library dependencies; therefore, every time a binary is analyzed, the number of true negatives is much higher (two orders of magnitude higher), causing other metrics to be imbalanced. Moreover, in our experiments, positive matches are more relevant than the negative ones.

6.3. Hyperparameters Selection

Before running our tests for library identification, we select the hyperparameters of our pipeline: `sim_thresh`, `min_size_cu`, and `min_cu_lib`. The first one, `sim_thresh`, is a direct consequence of the results discussed in Section 5. By evaluating the newly implemented Gemini-based GNN, we obtain the ROC-AUC curve shown in Fig. 2. This curve indicates that, when

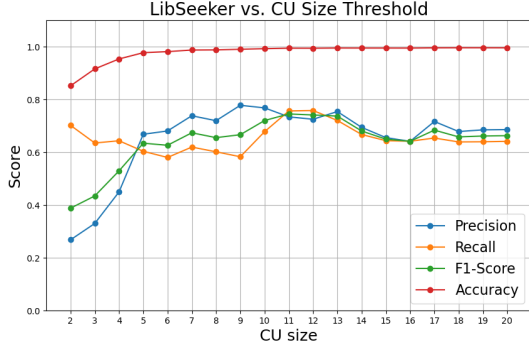


Figure 3. This plot shows how the metrics Precision, Recall, Accuracy, and F1 Score are affected by different CU sizes. The considered minimum size is 2, as a CU of size 1 would directly result in the function identification problem, which is out of the scope of this paper.

using the cosine distance between embeddings, we can set the threshold for declaring two functions identical to 0.99 without any degradation in performance.

Instead, we tune the other two hyperparameters by performing an empirical search, evaluating performance by varying `min_size_cu` and `min_cu_lib`. In short, these last two thresholds enable us to properly combine our model results with the proposed heuristics, thereby minimizing false positives and exploring the boundaries within which LibSeeker can perform optimally. Moreover, this selection process aims at confirming the validity of the matching algorithm described in Algorithm 1 and the granularity at which it is applied by assessing how well LibSeeker can determine whether a compilation unit has been included. This step assumes that our trained model can supply an initial candidate for matching a CU—an anchor. If one is provided, we proceed by applying the matching algorithm; otherwise, in the absence of such an anchor, the compilation unit under analysis is considered unmatched, as the starting point for our search is unknown, resulting in a too large search scope. The anchor is identified as the pair of functions with the highest similarity score, which must be at least above the threshold of 0.9. In this scenario, it is essential to note that the anchor itself may not always be accurate, as it might be incorrectly recognized by our model. Therefore, we evaluate how LibSeeker performs even under unreliable or noisy conditions, i.e., when faulty premises are provided to the matching algorithm. Specifically, we consider precision (P), recall (R), accuracy (A), and F1 score, with results detailed in Fig. 3. We consider a True Positive case when a library’s compilation unit is present (the reference baseline is identified via the binary’s symbols) and it is matched with a `sim_thresh` value higher than 0.99. Likewise, a True Negative case verifies when a library’s CU that has not been imported does not match any subset of functions within the binary with a high-enough similarity threshold.

As illustrated in Fig. 3, all metrics show a clear trend of improvement as the minimum CU size threshold is increased. This presents a trade-off: while a higher threshold improves the metrics’ results, it also reduces the number of CUs included in the analysis (i.e., the support). We select `min_size_cu` to 10 as a compromise that balances reliable performance with broad data coverage,

preventing the exclusion of too many smaller CUs. The high precision of our matching algorithm suggests that `min_cu_lib` can be set as low as 1, although a more complex interplay between these two thresholds could be a subject for future exploration. A final observation should be made about the Accuracy metric. As `min_size_cu` increases, accuracy trends toward 1. However, this should not be mistaken for a proportional improvement in the heuristic’s performance. Instead, it is an artifact of the changing relative weights of true negatives and true positives. As the support shrinks, these samples decrease in an imbalanced way, causing the TN portion to dominate the calculation. Thus, while the F1 score shows a more robust measure of performance, the accuracy score becomes an increasingly inflated and misleading metric.

The final hyperparameter configuration is as follows:

- `sim_thresh` set to 0.99, when the average similarity score for a compilation unit is equal to or higher than 0.99, the compilation is marked as matched.
- `min_size_cu` set to 10, smaller compilation units are considered to be unreliable if matched;
- `min_cu_lib` set to 1, it is enough to match one wide enough compilation unit to consider a library as matched;

With such a setup, a library is considered to be matched if there is at least one compilation unit (to be compliant with `min_cu_lib` threshold) that is large enough to be considered (accordingly to `min_size_cu`, it has to contain at least 10 functions) with an average similarity score higher than `sim_thresh`. This means that partial matches of functions belonging to the same compilation unit may not be sufficient to mark that CU as a match. The same reasoning can be applied to the library itself: matching some sparse functions, even with a very high similarity score, might not be enough to consider the entire library as matched. In other words, the markers for a library to be matched are compilation units as a whole rather than single functions, but such CUs must be big enough in terms of size. Accordingly, we will have True Positive samples when both our tool and the ground truth we defined mark a library as a dependency.

6.4. Library Identification (RQ1)

For the first research question, we use only the baseline version of each library (one of the three versions for each library, compiled beforehand with GCC-13 and the default optimization level, with the LTO optimization flag not set). From them, we extract the compilation units, and then we begin analyzing each binary to identify imported compilation units. We compare the obtained results against a proper ground truth (built with `ldd` support) to create a confusion matrix for each analyzed binary. We proceed by calculating the Precision, Recall, and F1 score, as reported in Table 2.

As we can see from the overall results, the Precision metric is quite high, indicating that our tool is effectively capable of identifying positive matches correctly; it hardly commits errors when marking a library as a dependency. At the same time, it may miss some true dependencies (false negative samples), as identified by the Recall metric. These results are a direct consequence of setting the

`sim_thresh` threshold to a high value, resulting in a more conservative model that prioritizes precision over recall. However, a good overall balance is maintained, as evidenced by an F1 score of 0.79.

In a more general way, all the thresholds we set beforehand affect these results. For instance, the `min_size_cu` threshold defines the set of compilation units to be used for matching, based on their size. So, libraries made by compilation units with a size smaller than the threshold might be detected, but, since untrusted, will never be marked as imported. On the contrary, however, if we considered all the compilation units as valid, despite their size, our results would have been negatively affected, generating a high number of false positives (although with a reduced number of false negatives). Therefore, the entire trade-off lies in finding the correct hyperparameters to tune the tool for the final scope, depending on how many false positives and false negatives are tolerated. So, depending on the use case, thresholds can be changed and adapted, prioritizing one metric over the others.

Comparing the results of this experiment with those presented in Fig. 3, we observe a slight improvement. This discrepancy is expected: library-level matching aggregates individual CU results, which can hide errors at the CU level, particularly for false negatives. To clarify, let us assume there is an imported library made with 10 compilation units large enough to be matched. Correctly matching only one CU would yield a single True Positive and nine False Negatives, leading to poor performance. In contrast, when performing library-level matching, as done in this experiment, identifying just one CU as a match is enough to classify the library as imported. This results in one True Positive and no additional errors. Consequently, a single True Positive at the library level can obscure many of the False Negatives at the CU level. In conclusion, ignoring the smaller-than-threshold compilation units and setting a high value for the similarity threshold helps our tool produce more reliable results, accepting a trade-off between false positives and false negatives. In the end, some smaller libraries might not be correctly identified, but the overall results are quite balanced.

Moreover, we attempt to gain a deeper understanding of how our tool behaves under different optimization levels, as shown in Table 2. There are no significant variations in the metrics we analyze when considering different optimization flags. Such a result is two-folded: on one side, it proves the capabilities of our tool in generalizing to unseen data, despite of the level of optimization (even considering the most aggressive ones, O3 and Os); on the other, it also reflects that compilation units are imported as they are, without any further optimization, despite the level of optimization adopted in when compiling the binary from source. This is the reason why the experiments are not affected by such a variable.

6.5. Library Version Identification (RQ2)

For the second research question, we use all the library versions available. We extract the compilation units from each library and then compare the binaries against them. Now, what we analyze is how well our tool performs in recognizing the exact library version, computing the same

TABLE 2. CLASSIFICATION METRICS SUMMARY FOR EXPERIMENT 1 (RQ1). THE TABLE ALSO REPORTS METRICS WHEN CONSIDERING DIFFERENT OPTIMIZATION LEVELS.

	Precision	Recall	F1
O0	0.86	0.70	0.78
O2	0.82	0.78	0.80
O3	0.83	0.78	0.80
Os	0.86	0.70	0.77
Overall	0.85	0.74	0.79

TABLE 3. CLASSIFICATION METRICS SUMMARY FOR EXPERIMENT 2 (RQ2). ROWS 2 AND 3 REFER TO THE SAME EXPERIMENT, BUT WE CONSIDER, RESPECTIVELY, A LIBRARY DATASET CONSISTING ONLY OF LIBRARIES OF THE NEAREST VERSION AND THE FURTHEST VERSION.

Support	Precision	Recall	F1
Full Suite	0.29	0.73	0.41
Minor Hop	0.60	0.73	0.66
Major Hop	0.56	0.70	0.62

metrics as before, but considering different versions of the same library as separate samples.

Table 3 reports our performances in three slightly different experimental settings. The first row—i.e., Full Suite—refers to the case in which we provide LibSeeker with all the library versions; the second row—i.e., Minor Hop—refers to the case in which we provide LibSeeker only with the library version nearest to the one actually used; the third row—i.e., Major Hop—refers to the case in which we provide LibSeeker only with the library version furthest from the one actually used.

Full Suite. Not surprisingly, when considering the full suite of library versions, the results we obtain exhibit a significant drop in precision, while recall remains roughly the same, as shown by the first column of Table 3. In this scenario, a True Positive occurs if and only if LibSeeker identifies an imported library with the exact same version; identifying a library with an incorrect version counts as a False Positive. Consequently, the F1 score drops to 0.41, primarily due to the lower precision. The performance drop is explained by how changes in the source code affect a compiled library. Some portion of a library might remain stable across different versions, even across major releases; therefore, if those unchanged CUs are the ones imported, it would be impossible to distinguish one version from another. Of course, the higher the distance in terms of versions from the used library, the higher the probability that changes in the source code affect all the compilation units, making it easier to distinguish between the two versions. Summarizing, sometimes identifying the correct version is impossible by design; yet, it can still be helpful to have hints about which library is a dependency and, eventually, what set of versions might have been used. Lastly, we evaluate how different optimization flags influence our performance. As shown in Table 4, our results remain unchanged compared to the ones of the previous experiment, reinforcing the idea that CUs are imported as-is, at least when the LTO flag is not involved.

Major and Minor. To gain a deeper insight into this experiment, we also examine the case in which only a subset of versions is available for each library. In this scenario, when LibSeeker is given only a single version

TABLE 4. CLASSIFICATION METRICS SUMMARY FOR EXPERIMENT 2 (RQ2), CONSIDERING DIFFERENT OPTIMIZATION LEVELS.

	Precision	Recall	F1
O0	0.29	0.70	0.41
O2	0.29	0.77	0.42
O3	0.29	0.76	0.41
Os	0.29	0.69	0.41
Overall	0.29	0.73	0.41

per library, we count a True Positive whenever it identifies an imported library, even if the actual version differs, as long as it matches the version we supplied. Not surprisingly, slicing our library dataset results in a performance boost, as shown in rows 2 and 3 of Table 3, particularly in terms of precision, which increases significantly due to the decrease in false positives—from 3227 to 859. Moreover, considering only a minor version downgrade yields slightly better performance, although LibSeeker still struggles to efficiently distinguish between different versions of the same library. The origin of this problem lies in how changes affect compilation units themselves, as already discussed. Especially from a reverse engineering perspective, it can strongly help identify the origin of each function. LibSeeker proves to be reliable enough, even across different versions, offering useful suggestions for classifying each function.

6.6. Cross-Compiler Library Identification (RQ3)

With this experiment, we aim to demonstrate the robustness of our work in a scenario where we are not aware of the specifics of how a library has been compiled. Namely, we compile the libraries from source using two different compilers, GCC-9 and GCC-13; then we recompile the dataset with the new set of libraries, the ones compiled with GCC-9. At last, as a test, we launch our pipeline trying to match libraries compiled with GCC-13.

Now, the goal of this experiment is to determine whether our heuristics are robust enough to generalize across different types of compilers. These compilers employ varying optimization strategies, resulting in distinct ways of compiling the same compilation unit, which in turn leads to different exported functions and varying sizes of compilation units. Table 5 shows that LibSeeker is not affected by the specifics of the compiler, corroborating the robustness of our matching algorithm.

The slight performance improvement over the library identification experiment in Section 6 can be attributed to the narrower setting considered here: in this experiment, LibSeeker is evaluated on a more restricted set of libraries since not all the libraries can be compiled with different compiler versions. While the smaller dataset introduces greater variance in the results, the observed improvement falls on the favorable side of this variability. Nonetheless, we regard the obtained metrics as comparable to those of the first experiment, confirming the consistency of our approach across both settings.

7. Related Works

Over the past several years, the problem of binary diffing has been examined using various approaches. The

TABLE 5. CLASSIFICATION METRICS SUMMARY FOR EXPERIMENT 3 (RQ3), CONSIDERING DIFFERENT OPTIMIZATION LEVELS.

	Precision	Recall	F1
O0	0.97	0.70	0.82
O2	0.93	0.78	0.85
O3	0.94	0.78	0.85
Os	0.97	0.70	0.81
Overall	0.95	0.74	0.83

core of this problem is finding a way to compare different pieces of code fairly and effectively, without access to the source code.

The well-known FLIRT [45] is a commercial plugin for IDA PRO [46]. It focuses on the problem of recognizing functions imported from third-party libraries, using the first bytes of each function as a signature for identification. This approach has several limitations, mostly coming from the pattern-matching method adopted, as even a slight change in those first bytes results in a false negative match. Moreover, it leads to the generation of many signature collisions whenever two functions have the same prologue. As a result, this approach is fast and practical, but unreliable, as acknowledged by the developers in the plugin’s documentation.

In 2017, Shirani et al. introduced BinShape [47], which attempts to extract a more valid and robust function signature. Leveraging static analysis, it derives function signatures based on a set of extracted features, combining statistical analysis with both CFG and instruction-level properties. The generated function signatures are then stored in an ad-hoc data structure for fast and reliable retrieval and comparison. During analysis, functions in the target binary are compared individually against those in the library to identify potential matches. However, a key limitation of this approach is its focus on function-level matching. In statically linked binaries, libraries are imported at the granularity of compilation units, meaning entire sets of functions are brought in together. By restricting the comparison to one function at a time, BinShape narrows its matching scope and fails to exploit the collective information present at the compilation unit level.

In 2020, Duan et al. presented DeepBinDiff [48], which introduces unsupervised program-wide code representation learning techniques that aim to combine the semantic information of assembly instructions with the contextual information from the CFG. This work, however, limits the granularity of their search to the basic block level: the comparison between pieces of code is carried out at a finer grain, thereby increasing flexibility and adaptability at the cost of efficiency. Post-analysis aggregation for function matching and, consequently, library matching, results in a significant drop in performance, lacks precision, and is unsuitable for large-scale deployment. Despite this, it is considered the de facto standard in commercial binary diffing. More recent works have continued along this path, moving toward new approaches that aid static analysis with neural networks.

The methodology proposed by Tang et al. with LibDb [49] uses manually extracted features to decorate the nodes of the CFGs of each function. These decorated graphs are then passed as input to Gemini [2] to generate

function-level embeddings that allow function-to-function similarity comparisons. Similarly, Mantilla [50], a recent work by Carrillo-Mondejar et al., uses static analyses to extract features for each function, which are then provided to a KNN classifier—combined with majority voting—to determine which libraries (along with their specific versions) have been imported in the statically linked binary file under analysis. However, statically extracted features (e.g., strings, number of basic blocks, instruction counts) cannot fully capture the semantics or contextual information present within the selected scope. In particular, manual feature extraction is a highly error-prone process inherently limited in its ability to recover all meaningful information.

Lastly, identification of inline library functions has been explored by Binosi et al. with BINO [29] and Highliner [51]. BINO detects inlined instances of C++ methods from both library and non-library template classes at a basic-block granularity, using a fingerprint-generation and matching approach. The matches produced by BINO are then further analysed by Highliner, which employs an NLP-based methodology to identify inlined instances at instruction-level granularity.

7.1. Existing Works Comparison

Comparison with the state of the art is always a key aspect for a correct validation of a project. Unfortunately, none of the previously cited works released their source code or the data upon which they operate, which presents a significant research problem, as it is not possible to conduct a meaningful and fair comparison. Even when considering the in-house reimplementations of the cited tools for comparison, several details would be missing from the paper description. The reproducibility of their experiments and results is compromised; therefore, comparisons on the same dataset would be meaningless, as they would be strongly affected by the approximated implementation that we could replicate. Nevertheless, we compare LibSeeker with prior works from a theoretical perspective, keeping in consideration the performance claimed in the respective papers.

The work most closely related to ours is BinShape [47]. In 2017, the authors reported an F1 score of 0.89 using a large set of statically extracted features, along with a promising ROC curve. However, their reported performance is heavily influenced by the composition of their experimental dataset: the library set they use for comparison is limited to the C library [52], which is almost always imported during compilation, resulting in a performance boost. A more rigorous experiment should include a larger and more diverse set of libraries, as the presence of less common libraries introduces more challenging edge cases for matching. In our experiments, we consider a sufficiently broad dataset of libraries, demonstrating that LibSeeker can generalize to recognize a variety of imported libraries rather than focusing solely on the most common ones. Consequently, we observe a slight drop in performance, with an F1 score of 0.79. However, for a fair comparison, BinShape would need to be evaluated on the same experimental dataset as LibSeeker. Furthermore, BinShape relies heavily on features extracted from static analysis. Advances in compilation techniques make code

diffing more challenging, as aggressive optimizations can significantly alter the extraction of features. In contrast, our innovative approach, based on compilation units rather than functions as isolated entities, achieves solid results and demonstrates robustness against different compilers and optimization strategies.

The lack of publicly available prototypes and datasets extends to other related work as well [3], [48]–[50]. Reported results in these studies are significantly weaker than those of BinShape and LibSeeker, although, once again, a fair comparison is impossible without a common evaluation dataset. To support future research and reproducibility in this area, we release the source code of our framework, a working prototype, and all datasets used in our experiments. Despite these limitations, we propose a qualitative evaluation of prior work, as shown in Table 6, categorizing the different steps involved in the library identification task to facilitate a meaningful comparison between approaches. Specifically, we distinguish three key dimensions: feature extraction, comparison strategy, and granularity level.

Feature Extraction. It’s the task by which a binary is analyzed to extract useful information, such as function boundaries, assembly instructions, etc. In addition to this information, it is possible to develop analyses to preprocess inputs for accurate comparisons. As Table 6 shows, all works are based on static analysis, from the simplest ones (e.g., CFG extraction) to more in-depth ones (e.g., cyclomatic complexity). Only BinShape differs from the others by applying feature extraction as well, accepting the compromise of boosting scalability and efficiency, at the cost of working on less information.

Comparison. It defines the way the search for the library’s functions or imports is carried out. As for pattern matching, the main limitation of this approach lies in its poor resilience to data variety, which strongly limits the applicability of FLIRT. On the other side, the application of machine learning techniques (e.g., embedders, graph neural networks) is more suited to generalize to new unseen data. Moreover, the set of required features, as well as the most relevant ones, is autonomously selected by the model that is chosen during the training stage. For the sake of completeness, a third option is also reported, referred to as the ‘Ad-Hoc Technique’, which involves the use of specifically developed data structures or algorithms.

Level of Granularity. It sets the scope at which the library search task is carried out, which, actually, is a non-secondary aspect to consider. A scope that is too small, such as function-level granularity, results in poor usability and performance, with frameworks being prone to errors due to mismatches, as seen with FLIRT and BinShape. On the other hand, considering the entire library altogether makes the matching task harder, especially since compilers import only the portion of functions that are actually used, as is the case with LibDb and Mantilla. For this reason, this work focuses on the **Compilation Unit** level, a middle ground in terms of scope size, with the advantage of truly mimicking how compilers and linkers behave.

TABLE 6. THIS TABLE SHOWS THE THREE MAIN BUILDING BLOCKS FOR LIBRARY IDENTIFICATION. DISASSEMBLING REFERS TO THE TECHNIQUE USED TO EXTRACT USEFUL FEATURES FROM A BINARY. COMPARISON REFERS TO THE TECHNIQUE USED TO IDENTIFY SIMILARITIES IN FUNCTIONS. MATCHING GRANULARITY REFERS TO THE SCOPE WITHIN WHICH LIBRARIES ARE SEARCHED WITHIN A STATICALLY LINKED BINARY.

	FEATURE EXTRACTION		COMPARISON			MATCHING GRANULARITY		
	Static Analysis	Feature Selection	Pattern-Matching	Ad-Hoc technique	ML\GNN	Function	CU	Library
FLIRT	•		•			•		
BinShape	•	•		•		•		
LibDb	•				•			•
Mantilla	•				•			•
LibSeeker	•			•	•		•	

8. Limitation and Future Works

The results shown in Section 6 demonstrate the validity of our approach based on the key concept that compilation units are the core of static libraries.

At the same time, it strongly depends on the size of compilation units, because it is the fundamental hyperparameter that defines the reliability of correctly identifying a library in a statically linked binary. As already discussed, when the number of functions in a CU is too small, then the amount of False Positives is greatly increased. Ideally, we would like to identify a lower bound for which a compilation should be considered, yet it depends on the imported functions as well, especially on their structure: functions that are too small are too difficult to be properly distinguished from each other.

Compilation units are designed to collect a group of functions, typically comprising at least two functions, which facilitates the search for a larger number of imported functions with more reliable results. In our approach, we select the hyperparameter `size_cu` to be 10, as the best trade-off between narrowing the set of compilation units considered for matching and the reliability of the output of LibSeeker.

Future work should move in this direction: the Hungarian algorithm [35] we adopt in our matching algorithm to search for a correspondence between a compilation unit and a window of functions is a fast and simple methodology, but it can be further enhanced. In our approach, there are two main assumptions: (i) the spatial organization of functions in a statically linked binary, meaning that all the imported functions are placed together, one next to the other, preserving the same order as in the compilation unit, (ii) the lack of function inlining, which allows us to look for a window of functions as wide as the compilation unit. Enabling the link-time optimization flag may weaken the assumptions made in this work; therefore, a more refined search of functions should be implemented, especially to consider inlined functions.

Ultimately, in our work, the match on the compilation unit is based on the average similarity between two sets of functions: the compilation unit itself and the best-matching window. However, more markers can be used. For instance, it is possible to analyze the bytes in the data section of both the binary and the compilation unit, checking for correspondences. In fact, importing a function implies importing the strings, the variables, and all the data upon which it depends at runtime. Therefore, such information can be considered as additional markers to achieve more reliable results. Moreover, it would also help reduce the `min_cu_size` threshold for compilation

units, as well as provide a better identification of library versions.

9. Conclusion

In this paper, we addressed the problem of recognizing imported libraries in statically compiled binaries, a task similar to well-known problems such as code similarity and binary diffing.

We introduce a novel approach rooted in the compilation units, since they are the core, atomic units of static libraries, allowing us to mimic how compilers and linkers operate. Working at a CU-level granularity, we implemented a graph neural network capable of capturing both the semantic and syntactic information of each function, generating meaningful embeddings for use in similarity comparisons. The pipeline used by LibSeeker in the library identification task is composed of three steps: (i) pre-processing, which extracts valuable embeddings from raw assembly instructions, (ii) embedding generation, which provides embeddings for each function, combining the semantics of instructions with the syntactics of CFGs, (iii) CU matching, which checks if a compilation unit is presents within the binary under analysis. At the end of this pipeline, if a compilation unit has been imported, then we can mark the corresponding library as a dependency.

To thoroughly and extensively evaluate LibSeeker’s capabilities, we conducted a set of experiments under various compilation scenarios, including multiple library versions and compilers. Experimental results demonstrate that LibSeeker performs very well under various conditions, confirming the effectiveness of adopting compilation units as the granularity level for the library identification task.

Acknowledgment

AI Use Disclosure. In accordance with the conference policy on AI-generated content, we disclose that AI-based tools were used solely for proofreading and grammar refinement during the preparation of this manuscript. No text, figures, images, code, or other scientific content was generated by AI systems. All conceptual, experimental, analytical, and written contributions remain the work of the authors.

References

- [1] A. M. Turing, “On computable numbers, with an application to the entscheidungsproblem,” *Proc. London Math. Soc.*, vol. s2-42, no. 1, pp. 230–265, 1937.

- [2] X. Xu, C. Liu, Q. Feng, H. Yin, L. Song, and D. Song, "Neural network-based graph embedding for cross-platform binary code similarity detection," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS 2017, Dallas, TX, USA, October 30 - November 03, 2017* (B. Thuraisingham, D. Evans, T. Malkin, and D. Xu, eds.), pp. 363–376, ACM, 2017.
- [3] Q. Feng, R. Zhou, C. Xu, Y. Cheng, B. Testa, and H. Yin, "Scalable graph-based bug search for firmware images," in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, Vienna, Austria, October 24-28, 2016* (E. R. Weippl, S. Katzenbeisser, C. Kruegel, A. C. Myers, and S. Halevi, eds.), pp. 480–491, ACM, 2016.
- [4] Y. Li, C. Gu, T. Dullien, O. Vinyals, and P. Kohli, "Graph matching networks for learning the similarity of graph structured objects," in *Proceedings of the 36th International Conference on Machine Learning, ICML 2019, 9-15 June 2019, Long Beach, California, USA* (K. Chaudhuri and R. Salakhutdinov, eds.), vol. 97 of *Proceedings of Machine Learning Research*, pp. 3835–3845, PMLR, 2019.
- [5] X. Li, Y. Qu, and H. Yin, "Palmtree: Learning an assembly language model for instruction embedding," in *CCS '21: 2021 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, Republic of Korea, November 15 - 19, 2021* (Y. Kim, J. Kim, G. Vigna, and E. Shi, eds.), pp. 3236–3251, ACM, 2021.
- [6] S. H. H. Ding, B. C. M. Fung, and P. Charland, "Asm2vec: Boosting static representation robustness for binary clone search against code obfuscation and compiler optimization," in *2019 IEEE Symposium on Security and Privacy, SP 2019, San Francisco, CA, USA, May 19-23, 2019*, pp. 472–489, IEEE, 2019.
- [7] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers)* (J. Burstein, C. Doran, and T. Solorio, eds.), pp. 4171–4186, Association for Computational Linguistics, 2019.
- [8] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," in *Advances in Neural Information Processing Systems 26: 27th Annual Conference on Neural Information Processing Systems 2013. Proceedings of a meeting held December 5-8, 2013, Lake Tahoe, Nevada, United States* (C. J. C. Burges, L. Bottou, Z. Ghahramani, and K. Q. Weinberger, eds.), pp. 3111–3119, 2013.
- [9] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu, "A comprehensive survey on graph neural networks," *CoRR*, vol. abs/1901.00596, 2019.
- [10] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," in *1st International Conference on Learning Representations, ICLR 2013, Scottsdale, Arizona, USA, May 2-4, 2013, Workshop Track Proceedings* (Y. Bengio and Y. LeCun, eds.), 2013.
- [11] J. Jang, S. Choi, and J. Hong, "A method for resilient graph-based comparison of executable objects," in *Research in Applied Computation Symposium, RACS '12, San Antonio, TX, USA, October 23-26, 2012* (Y. Cho, R. E. Gantenbein, T. Kuo, and V. Tarokh, eds.), pp. 288–289, ACM, 2012.
- [12] S. Yu, Y. G. Achamyeh, C. Wang, A. Kocheturov, P. Eisen, and M. A. A. Faruque, "CFG2VEC: hierarchical graph neural network for cross-architectural software reverse engineering," in *45th IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice, SEIP@ICSE 2023, Melbourne, Australia, May 14-20, 2023*, pp. 281–291, IEEE, 2023.
- [13] K. W. Church, "Word2vec," *Nat. Lang. Eng.*, vol. 23, no. 1, pp. 155–162, 2017.
- [14] D. Chicco, "Siamese neural networks: An overview," in *Artificial Neural Networks - Third Edition* (H. M. Cartwright, ed.), vol. 2190 of *Methods in Molecular Biology*, pp. 73–94, Springer, 2021.
- [15] M. Chandramohan, Y. Xue, Z. Xu, Y. Liu, C. Y. Cho, and H. B. K. Tan, "Bingo: cross-architecture cross-os binary search," in *Proceedings of the 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering, FSE 2016, Seattle, WA, USA, November 13-18, 2016* (T. Zimmermann, J. Cleland-Huang, and Z. Su, eds.), pp. 678–689, ACM, 2016.
- [16] Y. Hu, Y. Zhang, J. Li, and D. Gu, "Binary code clone detection across architectures and compiling configurations," in *Proceedings of the 25th International Conference on Program Comprehension, ICPC 2017, Buenos Aires, Argentina, May 22-23, 2017* (G. Scan-niello, D. Lo, and A. Serebrenik, eds.), pp. 88–98, IEEE Computer Society, 2017.
- [17] Z. Li, D. Zou, S. Xu, X. Ou, H. Jin, S. Wang, Z. Deng, and Y. Zhong, "Vuldeepecker: A deep learning-based system for vulnerability detection," in *25th Annual Network and Distributed System Security Symposium, NDSS 2018, San Diego, California, USA, February 18-21, 2018*, The Internet Society, 2018.
- [18] Y. Shoshitaishvili, R. Wang, C. Salls, N. Stephens, M. Polino, A. Dutcher, J. Grosen, S. Feng, C. Hauser, C. Krügel, and G. Vigna, "SOK: (state of) the art of war: Offensive techniques in binary analysis," in *IEEE Symposium on Security and Privacy, SP 2016, San Jose, CA, USA, May 22-26, 2016*, pp. 138–157, IEEE Computer Society, 2016.
- [19] Y. David, N. Partush, and E. Yahav, "Statistical similarity of binaries," in *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2016, Santa Barbara, CA, USA, June 13-17, 2016* (C. Kratz and E. D. Berger, eds.), pp. 266–280, ACM, 2016.
- [20] Y. David and E. Yahav, "Tracelet-based code search in executables," in *ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI '14, Edinburgh, United Kingdom - June 09 - 11, 2014* (M. F. P. O'Boyle and K. Pingali, eds.), pp. 349–360, ACM, 2014.
- [21] S. Eschweiler, K. Yakdan, and E. Gerhards-Padilla, "discover: Efficient cross-architecture identification of bugs in binary code," in *23rd Annual Network and Distributed System Security Symposium, NDSS 2016, San Diego, California, USA, February 21-24, 2016*, The Internet Society, 2016.
- [22] J. Powny, B. Garmany, R. Gawlik, C. Rossow, and T. Holz, "Cross-architecture bug search in binary executables," in *2015 IEEE Symposium on Security and Privacy, SP 2015, San Jose, CA, USA, May 17-21, 2015*, pp. 709–724, IEEE Computer Society, 2015.
- [23] V. van der Veen, E. Göktas, M. Contag, A. Pawlowski, X. Chen, S. Rawat, H. Bos, T. Holz, E. Athanasopoulos, and C. Giuffrida, "A tough call: Mitigating advanced code-reuse attacks at the binary level," in *IEEE Symposium on Security and Privacy, SP 2016, San Jose, CA, USA, May 22-26, 2016*, pp. 934–953, IEEE Computer Society, 2016.
- [24] M. Cova, V. Felmetger, G. Banks, and G. Vigna, "Static detection of vulnerabilities in x86 executables," in *22nd Annual Computer Security Applications Conference (ACSAC 2006), 11-15 December 2006, Miami Beach, Florida, USA*, pp. 269–278, IEEE Computer Society, 2006.
- [25] S. Alrabaee, P. Shirani, L. Wang, and M. Debbabi, "SIGMA: A semantic integrated graph matching approach for identifying reused functions in binary code," *Digit. Investig.*, vol. 12 Supplement 1, pp. S61–S71, 2015.
- [26] S. Alrabaee, L. Wang, and M. Debbabi, "BinGold: Towards robust binary analysis by extracting the semantics of binary code as semantic flow graphs (sfgs)," *Digit. Investig.*, vol. 18 Supplement, pp. S11–S22, 2016.
- [27] M. R. Farhadi, B. C. M. Fung, Y. B. Fung, P. Charland, S. Preda, and M. Debbabi, "Scalable code clone search for malware analysis," *Digit. Investig.*, vol. 15, pp. 46–60, 2015.
- [28] H. Huang, A. M. Youssef, and M. Debbabi, "Binsequence: Fast, accurate and scalable binary code reuse detection," in *Proceedings of the 2017 ACM on Asia Conference on Computer and Communications Security, AsiaCCS 2017, Abu Dhabi, United Arab Emirates, April 2-6, 2017* (R. Karri, O. Sinanoglu, A. Sadeghi, and X. Yi, eds.), pp. 155–166, ACM, 2017.

- [29] L. Binosi, M. Polino, M. Carminati, and S. Zanero, “BINO: automatic recognition of inline binary functions from template classes,” *Comput. Secur.*, vol. 132, p. 103312, 2023.
- [30] X. Ren, M. Ho, J. Ming, Y. Lei, and L. Li, “Unleashing the hidden power of compiler optimization on binary code difference: an empirical study,” in *PLDI ’21: 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation, Virtual Event, Canada, June 20-25, 2021* (S. N. Freund and E. Yahav, eds.), pp. 142–157, ACM, 2021.
- [31] A. Marcelli, M. Graziano, X. Ugarte-Pedrero, Y. Fratanonio, M. Mansouri, and D. Balzarotti, “How machine learning is solving the binary function similarity problem,” in *31st USENIX Security Symposium, USENIX Security 2022, Boston, MA, USA, August 10-12, 2022* (K. R. B. Butler and K. Thomas, eds.), pp. 2099–2116, USENIX Association, 2022.
- [32] X. Shang, S. Cheng, G. Chen, Y. Zhang, L. Hu, X. Yu, G. Li, W. Zhang, and N. Yu, “How far have we gone in binary code understanding using large language models,” in *IEEE International Conference on Software Maintenance and Evolution, ICSME 2024, Flagstaff, AZ, USA, October 6-11, 2024*, pp. 1–12, IEEE, 2024.
- [33] X. Jin, J. Larson, W. Yang, and Z. Lin, “Binary code summarization: Benchmarking chatgpt/gpt-4 and other large language models,” *CoRR*, vol. abs/2312.09601, 2023.
- [34] X. Shang, G. Chen, S. Cheng, B. Wu, L. Hu, G. Li, W. Zhang, and N. Yu, “Binmetric: A comprehensive binary code analysis benchmark for large language models,” in *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence, IJCAI 2025, Montreal, Canada, August 16-22, 2025*, pp. 7715–7723, ijcai.org, 2025.
- [35] H. W. Kuhn, “The hungarian method for the assignment problem,” in *50 Years of Integer Programming 1958-2008 - From the Early Years to the State-of-the-Art* (M. Jünger, T. M. Lieblich, D. Naddef, G. L. Nemhauser, W. R. Pulleyblank, G. Reinelt, G. Rinaldi, and L. A. Wolsey, eds.), pp. 29–47, Springer, 2010.
- [36] Y. J. Lee, S.-H. Choi, C. Kim, S.-H. Lim, and K.-W. Park, “Learning binary code with deep learning to detect software weakness,” in *KSII the 9th international conference on internet (ICONI) 2017 symposium*, 2017.
- [37] M. J. Zaki, W. Meira Jr, and W. Meira, *Data mining and machine learning: fundamental concepts and algorithms*. Cambridge University Press, 2020.
- [38] D. Peng, Z. Gui, and H. Wu, “Interpreting the curse of dimensionality from distance concentration and manifold effect,” *CoRR*, vol. abs/2401.00422, 2024.
- [39] GNU, “Index of /gnu/binutils,” <https://www.gnu.org/software/binutils/>, 2025.
- [40] GNU, “Index of /gnu/coreutils,” <https://www.gnu.org/software/coreutils/>, 2025.
- [41] GNU, “Index of /gnu/findutils,” <https://www.gnu.org/software/findutils/>, 2025.
- [42] GNU, “Index of /gnu/libc,” <https://www.gnu.org/software/libc/>, 2025.
- [43] OpenBSD Project, “Openssh,” <https://www.openssh.org/>, 2025.
- [44] OpenSSL Software Foundation, “Openssl,” <https://www.openssl.org/>, 2025.
- [45] Hex-Rays, “Ida f.l.i.r.t. technology,” https://www.hex-rays.com/products/ida/tech/flirt/in_depth.shtml, 2025.
- [46] Hex-Rays, “Ida pro,” <https://docs.hex-rays.com/>, 2025.
- [47] P. Shirani, L. Wang, and M. Debbabi, “Binshape: Scalable and robust binary library function identification using function shape,” in *Detection of Intrusions and Malware, and Vulnerability Assessment - 14th International Conference, DIMVA 2017, Bonn, Germany, July 6-7, 2017, Proceedings* (M. Polychronakis and M. Meier, eds.), vol. 10327 of *Lecture Notes in Computer Science*, pp. 301–324, Springer, 2017.
- [48] Y. Duan, X. Li, J. Wang, and H. Yin, “Deepbindiff: Learning program-wide code representations for binary diffing,” in *27th Annual Network and Distributed System Security Symposium, NDSS 2020, San Diego, California, USA, February 23-26, 2020*, The Internet Society, 2020.
- [49] W. Tang, Y. Wang, H. Zhang, S. Han, P. Luo, and D. Zhang, “Libdb: An effective and efficient framework for detecting third-party libraries in binaries,” in *19th IEEE/ACM International Conference on Mining Software Repositories, MSR 2022, Pittsburgh, PA, USA, May 23-24, 2022*, pp. 423–434, ACM, 2022.
- [50] J. C. Mondéjar and R. J. Rodríguez, “Identifying runtime libraries in statically linked linux binaries,” *Future Gener. Comput. Syst.*, vol. 164, p. 107602, 2025.
- [51] L. Dall’Aglio, L. Binosi, M. Carminati, S. Zanero, and M. Polino, “Highliner: Enhancing binary analysis through nlp-based instruction-level detection of c++ inline functions,” *ACM Transactions on Privacy and Security*, vol. 28, no. 4, pp. 1–22, 2025.
- [52] C. L. Library, “Index of c language library,” <https://cplusplus.com/reference/clibrary/>.

Appendix A. Data Availability

In accordance with the conference Open Science Expectations, we aim to maximize transparency, reproducibility, and community value. We will publicly release LibSeeker upon publication under an open-source license upon acceptance. The source code is accessible here: <https://anonymous.4open.science/r/LibSeeker-F35D>. We also release two datasets: one for training and testing our model, and the other for experimental evaluation. Together with the already pre-compiled binaries and libraries, we also release their source code and repositories to enable future recompilation with different compilers and settings. The datasets are available here: https://osf.io/4x5eh/overview?view_only=ee92e2d75b9f40f9a09790ddc0ff29ff, <https://mega.nz/folder/e5JDnT7C#NZAw0siKTL7Ko6psjpt-4Q>. Lastly, we also release a pre-trained model, which is used in this paper for testing and experiments, and can be found at the following URL: https://osf.io/4x5eh/overview?view_only=ee92e2d75b9f40f9a09790ddc0ff29ff.

Appendix B. ML Theory

Word embeddings. Embeddings are dense, distributed, fixed-length word vectors built using word co-occurrence statistics following the distributional hypothesis, providing a means of representing words as numerical vectors that capture semantic and syntactic relationships within sentences. Several neural network models are trained for this task, specifically adapted to natural language processing; the most famous ones are BERT [7] and Word2Vec [8].

Graph Neural Networks. Graph Neural Network (GNN) are a type of neural network capable of working directly on graph-represented data. Graphs are complex, compact data structures that convey a significant amount of information. In fact, the number of layers of information rapidly increases due to connections: each node is carrying its own information, along with some information derived from its incoming and outgoing edges, namely from its neighbors; thus, in the end, the layout of a graph cannot be ignored. The goal of a GNN is to combine both these characteristics: the semantic information of each node and the structural information provided by the edges, to build upon them. To properly work, such models require

extracting features that can semantically characterize the nodes of the graph, and then, combining such pieces of information accordingly to the flow of edges to fuse them with the syntactic structure of the graph.

Appendix C.

Metrics Vademecum

For the sake of completeness, here we report the formulas of the metrics used in this paper. Please note that in the formulas we use abbreviations, specifically: True Positives (TP), False Positives (FP), False Negatives (FN), and True Negatives (TN).

$$Precision = \frac{TP}{TP + FP} \quad (2)$$

$$Recall = \frac{TP}{TP + FN} \quad (3)$$

$$Accuracy = \frac{TP + TN}{TP + FP + FN + TN} \quad (4)$$

$$F1_score = \frac{2 \times Precision \times Recall}{Precision + Recall} \quad (5)$$